

¿Qué condiciones puedo hacer dentro de un if?

Ya hablamos de cómo es la estructura de un if y también dónde debe de ir la condición, pero, ¿Qué podemos ocupar como condición?

Bien, aquí tenemos 3 maneras de hacerlo y daré un ejemplo por cada uno.

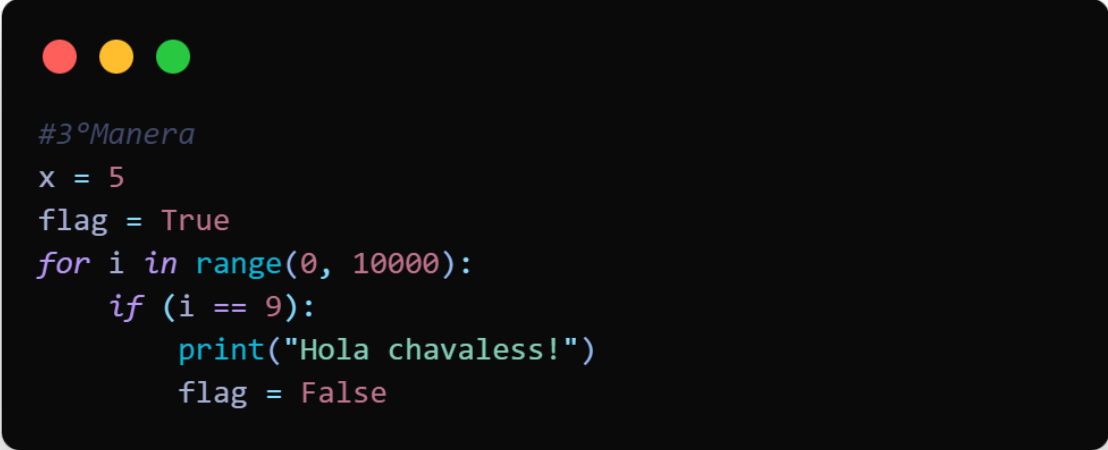
```
● ● ●  
  
#1ªManera  
x = 5 + 4  
y = 9  
  
if (x == y):  
    print("Hola chavaless!")
```

En nuestra primera forma, tenemos 2 variables independientes que son iguales (si fueran distintos igual sirve para este ejemplo, pero mejor tener algo verdadero primero) por lo que, dentro de **nuestro condicional decimos “si x es igual que y, ejecutaremos esto”**, y para efectos más “abstractos”, nuestros condicionales solo los veremos como verdadero o falso. En este caso, decimos “si 9 es igual que 9 ejecutaremos esto” ya que estamos comparando 2 variables (y al hacer esto, se comparan los valores que estas variables tienen, da igual si es de tipo str o int, mientras las comparaciones sean del mismo tipo funcionará).

```
● ● ●  
  
#2ªManera  
x = 5 + 4  
if (x == 9):  
    print("Hola chavaless!")
```

En nuestra segunda forma no estamos comparando 2 variables como tal, sino que estamos comparando algo dentro de la misma condición, es una de las formas que más se ocupan y

abre paso a “acortar” el código ya que no dependemos de una variable externa para hacer funcionar nuestra condición.



```
#3ªManera
x = 5
flag = True
for i in range(0, 10000):
    if (i == 9):
        print("Hola chavaless!")
        flag = False
```

En esta tercera manera de ocupar una condición podemos “Trucar” un poco los ciclos, ya que nuestra condicional siempre será verdadera hasta que NOSOTROS le digamos lo contrario, esta estructura sirve ya que dentro del curso es raro que se pase el comando “Break” que detiene abruptamente un ciclo, pero siempre se puede hacer esto (poner nuestra bandera como verdadera y dentro de un ciclo, un condicional donde nuestra condición sea la que queremos “trucar”). Generalmente les recomiendo hacer esto cuando tengan que recorrer (por ejemplo) grandes diccionarios y/o grandes listas con un ciclo for donde solamente deben extraer 1 dato, así van comparando hasta llegar a ese dato y pueden dejar de iterar.

En este ejemplo, **Iterar = ir cambiando nuestra variable repetidas veces, puede ser para recorrer una lista o simplemente ir sumando / restando algo (inclusive para cambiar una letra).**

Tip: Dentro de ciclos, acostumbrense a ocupar más ciclos for, intentar hacer las cosas con for les da la posibilidad de no iterar más de lo necesario. Un ciclo while puede ser más beneficioso cuando necesita repetir algo hasta que 1 sola condición se cumpla (maximo 2 o 3), pero si ya tienen una lista y necesitan hacer cosas dentro de ESA lista, mejor iteren con un ciclo for, así no tienen problemas de out of index.

Pero, ¿Qué operadores lógicos o de comparación podemos ocupar?

Bien, esto es más sistemático, escribir todas sería bastante aburrido y probablemente me faltaría 1 o 2 así que les dejo las listas:

Operador	Descripción	Ejemplo
<code>==</code>	Igual que	<code>5 == 5</code> → <code>True</code>
<code>!=</code>	Diferente que	<code>4 != 2</code> → <code>True</code>
<code><</code>	Menor que	<code>3 < 8</code> → <code>True</code>
<code><=</code>	Menor o igual que	<code>6 <= 6</code> → <code>True</code>
<code>></code>	Mayor que	<code>10 > 5</code> → <code>True</code>
<code>>=</code>	Mayor o igual que	<code>7 >= 8</code> → <code>False</code>

Esta es la lista de **operadores de comparación** y siempre debe ocuparse en casos como el **método 1 o 2** (A excepción del operador “==”, ya que este es el único que sí puede dar respuestas de verdadero o falso directamente y no depende de 2 variables o 1 variable y 1 operación).

Operador	Descripción	Ejemplo	Explicación
<code>and</code>	Verdadero si ambas condiciones son verdaderas	<code>(5 > 3) and (2 < 4)</code> → <code>True</code>	Se puede leer como “se cumple la <i>condición 1</i> y se cumple la <i>condición 2</i> ”.
<code>or</code>	Verdadero si al menos una condición es verdadera	<code>(5 > 3) or (2 > 4)</code> → <code>True</code>	Se puede leer como “se cumple la <i>condición 1</i> o se cumple la <i>condición 2</i> ”.
<code>not</code>	Invierte el valor de la condición	<code>not (5 > 3)</code> → <code>False</code>	Se puede leer como “ no se cumple la <i>condición</i> ”.

Estos son los operadores lógicos, probablemente ya hayan ocupado el operador “or” y “and” pero, ¿han visto el operador **not**?

como su descripción indica, invierte el valor de la condición, esto quiere decir que **todo lo que se considere verdadero se tomará como falso y viceversa**, a diferencia del operador “!=” que sirve para negar algo dentro de la condición, esto negará TODO lo que se encuentre fuera de la operación (básicamente, “!=” niega algo en la operación y “not” niega la operación misma).